

```
cout << "Your hex number is: " << result << endl;
```

假设输入的内容如下：

```
12 0 5 15 8 15
```

程序的输出结果将是：

```
Your hex number is: C05F8F
```

上述程序的执行过程是这样的：首先初始化变量 `hexdigits` 令其存放从 0 到 F 的十六进制数字，注意我们把 `hexdigits` 声明成了常量（参见 2.4 节，第 53 页），这是因为在后面的程序中不打算再改变它的值。在循环内部使用输入值 `n` 作为 `hexdigits` 的下标，`hexdigits[n]` 的值就是 `hexdigits` 内位置 `n` 处的字符。例如，如果 `n` 是 15，则结果是 F；如果 `n` 是 12，则结果是 C，以此类推。把得到的十六进制数字添加到 `result` 内，最后一并输出。

无论何时用到字符串的下标，都应该注意检查其合法性。在上面的程序中，下标 `n` 是 `string::size_type` 类型，也就是无符号类型，所以 `n` 可以确保大于或等于 0。在实际使用时，还需检查 `n` 是否小于 `hexdigits` 的长度。

96

3.2.3 节练习

练习 3.6：编写一段程序，使用范围 `for` 语句将字符串内的所有字符用 `x` 代替。

练习 3.7：就上一题完成的程序而言，如果将循环控制变量的类型设为 `char` 将发生什么？先估计一下结果，然后实际编程进行验证。

练习 3.8：分别用 `while` 循环和传统的 `for` 循环重写第一题的程序，你觉得哪种形式更好呢？为什么？

练习 3.9：下面的程序有何作用？它合法吗？如果不合法，为什么？

```
string s;
cout << s[0] << endl;
```

练习 3.10：编写一段程序，读入一个包含标点符号的字符串，将标点符号去除后输出字符串剩余的部分。

练习 3.11：下面的范围 `for` 语句合法吗？如果合法，`c` 的类型是什么？

```
const string s = "Keep out!";
for (auto& c : s) { /* ... */ }
```



3.3 标准库类型 `vector`

标准库类型 `vector` 表示对象的集合，其中所有对象的类型都相同。集合中的每个对象都有一个与之对应的索引，索引用于访问对象。因为 `vector` “容纳着”其他对象，所以它也常被称作容器（container）。第 II 部将对容器进行更为详细的介绍。

要想使用 `vector`，必须包含适当的头文件。在后续的例子中，都将假定做了如下 `using` 声明：

```
#include <vector>
using std::vector;
```

C++语言既有类模板（class template），也有函数模板，其中 `vector` 是一个类模板。只有对 C++有了相当深入的理解才能写出模板，事实上，我们直到第 16 章才会学习如何自定义模板。幸运的是，即使还不会创建模板，我们也可以先试着用用它。

模板本身不是类或函数，相反可以将模板看作为编译器生成类或函数编写的一份说明。编译器根据模板创建类或函数的过程称为实例化（instantiation），当使用模板时，需要指出编译器应把类或函数实例化成何种类型。

对于类模板来说，我们通过提供一些额外信息来指定模板到底实例化成什么样的类，需要提供哪些信息由模板决定。提供信息的方式总是这样：即在模板名字后面跟一对尖括号，在括号内放上信息。

以 `vector` 为例，提供的额外信息是 `vector` 内所存放对象的类型：

```
vector<int> ivec;           // ivec 保存 int 类型的对象
vector<Sales_item> Sales_vec; // 保存 Sales_item 类型的对象
vector<vector<string>> file; // 该向量的元素是 vector 对象
```

97

在上面的例子中，编译器根据模板 `vector` 生成了三种不同的类型：`vector<int>`、`vector<Sales_item>`和 `vector<vector<string>>`。

Note `vector` 是模板而非类型，由 `vector` 生成的类型必须包含 `vector` 中元素的类型，例如 `vector<int>`。

`vector` 能容纳绝大多数类型的对象作为其元素，但是因为引用不是对象（参见 2.3.1 节，第 45 页），所以不存在包含引用的 `vector`。除此之外，其他大多数（非引用）内置类型和类类型都可以构成 `vector` 对象，甚至组成 `vector` 的元素也可以是 `vector`。

需要指出的是，在早期版本的 C++标准中如果 `vector` 的元素还是 `vector`（或者其他模板类型），则其定义的形式与现在的 C++11 新标准略有不同。过去，必须在外层 `vector` 对象的右尖括号和其元素类型之间添加一个空格，如应该写成 `vector<vector<int> >` 而非 `vector<vector<int>>`。

C++ 11

WARNING 某些编译器可能仍需以老式的声明语句来处理元素为 `vector` 的 `vector` 对象，如 `vector<vector<int> >`。

3.3.1 定义和初始化 vector 对象



和任何一种类型一样，`vector` 模板控制着定义和初始化向量的方法。表 3.4 列出了定义 `vector` 对象的常用方法。

表 3.4：初始化 vector 对象的方法	
<code>vector<T> v1</code>	<code>v1</code> 是一个空 <code>vector</code> ，它潜在的元素是 <code>T</code> 类型的，执行默认初始化
<code>vector<T> v2(v1)</code>	<code>v2</code> 中包含有 <code>v1</code> 所有元素的副本
<code>vector<T> v2 = v1</code>	等价于 <code>v2(v1)</code> ， <code>v2</code> 中包含有 <code>v1</code> 所有元素的副本
<code>vector<T> v3(n, val)</code>	<code>v3</code> 包含了 <code>n</code> 个重复的元素，每个元素的值都是 <code>val</code>
<code>vector<T> v4(n)</code>	<code>v4</code> 包含了 <code>n</code> 个重复地执行了值初始化的对象
<code>vector<T> v5{a,b,c...}</code>	<code>v5</code> 包含了初始值个数的元素，每个元素被赋予相应的初始值
<code>vector<T> v5={a,b,c...}</code>	等价于 <code>v5{a,b,c...}</code>

可以默认初始化 `vector` 对象（参见 2.2.1 节，第 40 页），从而创建一个指定类型的空 `vector`：

```
vector<string> svec; // 默认初始化，svec 不含任何元素
```

看起来空 `vector` 好像没什么用，但是很快我们就会知道程序在运行时可以很高效地往 `vector` 对象中添加元素。事实上，最常见的方式就是先定义一个空 `vector`，然后当运行时获取到元素的值后再逐一添加。

当然也可以在定义 `vector` 对象时指定元素的初始值。例如，允许把一个 `vector` 对象的元素拷贝给另外一个 `vector` 对象。此时，新 `vector` 对象的元素就是原 `vector` 对象对应元素的副本。注意两个 `vector` 对象的类型必须相同：

```
vector<int> ivec;                // 初始状态为空
// 在此处给 ivec 添加一些值
vector<int> ivec2(ivec);        // 把 ivec 的元素拷贝给 ivec2
vector<int> ivec3 = ivec;       // 把 ivec 的元素拷贝给 ivec3
vector<string> svec(ivec2);     // 错误：svec 的元素是 string 对象，不是 int
```

98 列表初始化 `vector` 对象

C++
11

C++11 新标准还提供了另外一种为 `vector` 对象的元素赋初值的方法，即列表初始化（参见 2.2.1 节，第 39 页）。此时，用花括号括起来的 0 个或多个初始元素值被赋给 `vector` 对象：

```
vector<string> articles = {"a", "an", "the"};
```

上述 `vector` 对象包含三个元素：第一个是字符串"a"，第二个是字符串"an"，最后一个字符串是"the"。

之前已经讲过，C++语言提供了几种不同的初始化方式（参见 2.2.1 节，第 39 页）。在大多数情况下这些初始化方式可以相互等价地使用，不过也并非一直如此。目前已经介绍过的两种例外情况是：其一，使用拷贝初始化时（即使用`=`时）（参见 3.2.1 节，第 76 页），只能提供一个初始值；其二，如果提供的是一个类内初始值（参见 2.6.1 节，第 64 页），则只能使用拷贝初始化或使用花括号的形式初始化。第三种特殊的要求是，如果提供的是初始元素值的列表，则只能把初始值都放在花括号里进行列表初始化，而不能放在圆括号里：

```
vector<string> v1{"a", "an", "the"}; // 列表初始化
vector<string> v2("a", "an", "the"); // 错误
```

创建指定数量的元素

还可以用 `vector` 对象容纳的元素数量和所有元素的统一初始值来初始化 `vector` 对象：

```
vector<int> ivec(10, -1); // 10 个 int 类型的元素，每个都被初始化为 -1
vector<string> svec(10, "hi!"); // 10 个 string 类型的元素，
// 每个都被初始化为 "hi!"
```

值初始化

通常情况下，可以只提供 `vector` 对象容纳的元素数量而不用略去初始值。此时库会创建一个值初始化的（value-initialized）元素初值，并把它赋给容器中的所有元素。这个初值由 `vector` 对象中元素的类型决定。

如果 `vector` 对象的元素是内置类型，比如 `int`，则元素初始值自动设为 0。如果元素是某种类型，比如 `string`，则元素由类默认初始化：


```
vector<int> ivec(10);           // 10 个元素，每个都初始化为 0
vector<string> svec(10);       // 10 个元素，每个都是空 string 对象
```

对这种初始化的方式有两个特殊限制：其一，有些类要求必须明确地提供初始值（参见 2.2.1 节，第 40 页），如果 vector 对象中元素的类型不支持默认初始化，我们就必须提供初始的元素值。对这种类型的对象来说，只提供元素的数量而不设定初始值无法完成初始化工作。

其二，如果只提供了元素的数量而没有设定初始值，只能使用直接初始化：

```
vector<int> vi = 10; // 错误：必须使用直接初始化的形式指定向量大小
```

99

这里的 10 是用来说明如何初始化 vector 对象的，我们用它的本意是想创建含有 10 个值初始化了的元素的 vector 对象，而非把数字 10 “拷贝”到 vector 中。因此，此时不宜使用拷贝初始化，7.5.4 节（第 265 页）将对这一点做更详细的介绍。

列表初始值还是元素数量？



在某些情况下，初始化的真实含义依赖于传递初始值时用的是花括号还是圆括号。例如，用一个整数来初始化 vector<int> 时，整数的含义可能是 vector 对象的容量也可能是元素的值。类似的，用两个整数来初始化 vector<int> 时，这两个整数可能一个是 vector 对象的容量，另一个是元素的初值，也可能它们是容量为 2 的 vector 对象中两个元素的初值。通过使用花括号或圆括号可以区分上述这些含义：

```
vector<int> v1(10);           // v1 有 10 个元素，每个的值都是 0
vector<int> v2{10};           // v2 有 1 个元素，该元素的值是 10

vector<int> v3(10, 1);        // v3 有 10 个元素，每个的值都是 1
vector<int> v4{10, 1};        // v4 有 2 个元素，值分别是 10 和 1
```

如果用的是圆括号，可以说提供的值是用来构造（construct）vector 对象的。例如，v1 的初始值说明了 vector 对象的容量；v3 的两个初始值则分别说明了 vector 对象的容量和元素的初值。

如果用的是花括号，可以表述成我们想列表初始化（list initialize）该 vector 对象。也就是说，初始化过程会尽可能地把花括号内的值当成是元素初始值的列表来处理，只有在无法执行列表初始化时才会考虑其他初始化方式。在上例中，给 v2 和 v4 提供的初始值都能作为元素的值，所以它们都会执行列表初始化，vector 对象 v2 包含一个元素而 vector 对象 v4 包含两个元素。

另一方面，如果初始化时使用了花括号的形式但是提供的值又不能用来列表初始化，就要考虑用这样的值来构造 vector 对象了。例如，要想列表初始化一个含有 string 对象的 vector 对象，应该提供能赋给 string 对象的初值。此时不难区分到底是要列表初始化 vector 对象的元素还是用给定的容量值来构造 vector 对象：

```
vector<string> v5{"hi"};      // 列表初始化：v5 有一个元素
vector<string> v6("hi");      // 错误：不能使用字符串字面值构建 vector 对象
vector<string> v7{10};        // v7 有 10 个默认初始化的元素
vector<string> v8{10, "hi"};   // v8 有 10 个值为 "hi" 的元素
```

100

尽管在上面的例子中除了第二条语句之外都用了花括号，但其实只有 v5 是列表初始化。要想列表初始化 vector 对象，花括号里的值必须与元素类型相同。显然不能用 int 初始化 string 对象，所以 v7 和 v8 提供的值不能作为元素的初始值。确认无法执行列表初始化后，编译器会尝试用默认值初始化 vector 对象。

3.3.1 节练习

练习 3.12: 下列 `vector` 对象的定义有不正确的吗？如果有，请指出来。对于正确的，描述其执行结果；对于不正确的，说明其错误的原因。

- (a) `vector<vector<int>> ivec;`
- (b) `vector<string> svec = ivec;`
- (c) `vector<string> svec(10, "null");`

练习 3.13: 下列的 `vector` 对象各包含多少个元素？这些元素的值分别是多少？

- (a) `vector<int> v1;`
- (b) `vector<int> v2(10);`
- (c) `vector<int> v3(10, 42);`
- (d) `vector<int> v4{10};`
- (e) `vector<int> v5{10, 42};`
- (f) `vector<string> v6{10};`
- (g) `vector<string> v7{10, "hi"};`



3.3.2 向 `vector` 对象中添加元素

对 `vector` 对象来说，直接初始化的方式适用于三种情况：初始值已知且数量较少、初始值是另一个 `vector` 对象的副本、所有元素的初始值都一样。然而更常见的情况是：创建一个 `vector` 对象时并不清楚实际所需的元素个数，元素的值也经常无法确定。还有些时候即使元素的初值已知，但如果这些值总量较大而各不相同，那么在创建 `vector` 对象的时候执行初始化操作也会显得过于烦琐。

举个例子，如果想创建一个 `vector` 对象令其包含从 0 到 9 共 10 个元素，使用列表初始化的方法很容易做到这一点；但如果 `vector` 对象包含的元素是从 0 到 99 或者从 0 到 999 呢？这时通过列表初始化把所有元素都一一罗列出来就不太合适了。对于此例来说，更好的处理方法是先创建一个空 `vector`，然后在运行时再利用 `vector` 的成员函数 `push_back` 向其中添加元素。`push_back` 负责把一个值当成 `vector` 对象的尾元素“压到 (`push`)” `vector` 对象的“尾端 (`back`)”。例如：

101

```
vector<int> v2;           // 空 vector 对象
for (int i = 0; i != 100; ++i)
    v2.push_back(i); // 依次把整数值放到 v2 尾端
// 循环结束后 v2 有 100 个元素，值从 0 到 99
```

在上例中，尽管知道 `vector` 对象最后会包含 100 个元素，但在一开始还是把它声明成空 `vector`，在每次迭代时才顺序地把下一个整数作为 `v2` 的新元素添加给它。

同样的，如果直到运行时才能知道 `vector` 对象中元素的确切个数，也应该使用刚刚这种方法创建 `vector` 对象并为其赋值。例如，有时需要实时读入数据然后将其赋予 `vector` 对象：

```
// 从标准输入中读取单词，将其作为 vector 对象的元素存储
string word;
vector<string> text;           // 空 vector 对象
while (cin >> word) {
    text.push_back(word);      // 把 word 添加到 text 后面
}
```

和之前的例子一样，本例也是先创建一个空 `vector`，之后依次读入未知数量的值并保存到 `text` 中。

关键概念: vector 对象能高效增长

C++标准要求 vector 应该能在运行时高效快速地添加元素。因此既然 vector 对象能高效地增长,那么在定义 vector 对象的时候设定其大小也就没什么必要了,事实上如果这么做性能可能更差。只有一种例外情况,就是所有(all)元素的值都一样。一旦元素的值有所不同,更有效的办法是先定义一个空的 vector 对象,再在运行时向其中添加具体值。此外,9.4 节(第 317 页)将介绍, vector 还提供了方法,允许我们进一步提升动态添加元素的性能。

开始的时候创建空的 vector 对象,在运行时再动态添加元素,这一做法与 C 语言及其他大多数语言中内置数组类型的用法不同。特别是如果用惯了 C 或者 Java,可以预计在创建 vector 对象时顺便指定其容量是最好的。然而事实上,通常的情况是恰恰相反。

向 vector 对象添加元素蕴含的编程假定

由于能高效便捷地向 vector 对象中添加元素,很多编程工作被极大简化了。然而,这种简便性也伴随着一些对编写程序更高的要求:其中一条就是必须要确保所写的循环正确无误,特别是在循环有可能改变 vector 对象容量的时候。

随着对 vector 的更多使用,我们还会逐渐了解到其他一些隐含的要求,其中一条是现在就要指出的:如果循环体内部包含有向 vector 对象添加元素的语句,则不能使用范围 for 循环,具体原因将在 5.4.3 节(第 168 页)详细解释。



范围 for 语句体内不应改变其所遍历序列的大小。

3.3.2 节练习

练习 3.14: 编写一段程序,用 cin 读入一组整数并把它们存入一个 vector 对象。

练习 3.15: 改写上题的程序,不过这次读入的是字符串。

3.3.3 其他 vector 操作



除了 push_back 之外, vector 还提供了几种其他操作,大多数都和 string 的相关操作类似,表 3.5 列出了其中比较重要的一些。

表 3.5: vector 支持的操作	
v.empty()	如果 v 不含有任何元素,返回真;否则返回假
v.size()	返回 v 中元素的个数
v.push_back(t)	向 v 的尾端添加一个值为 t 的元素
v[n]	返回 v 中第 n 个位置上元素的引用
v1 = v2	用 v2 中元素的拷贝替换 v1 中的元素
v1 = {a,b,c...}	用列表中元素的拷贝替换 v1 中的元素
v1 == v2	v1 和 v2 相等当且仅当它们的元素数量相同且对应位置的元素值都相同
v1 != v2	
<, <=, >, >=	顾名思义,以字典顺序进行比较

访问 `vector` 对象中元素的方法和访问 `string` 对象中字符的方法差不多,也是通过元素在 `vector` 对象中的位置。例如,可以使用范围 `for` 语句处理 `vector` 对象中的所有元素:

```
vector<int> v{1,2,3,4,5,6,7,8,9};
for (auto &i : v)           // 对于v中的每个元素(注意: i 是一个引用)
    i *= i;                 // 求元素值的平方
for (auto i : v)           // 对于v中的每个元素
    cout << i << " ";     // 输出该元素
cout << endl;
```

第一个循环把控制变量 `i` 定义成引用类型,这样就能通过 `i` 给 `v` 的元素赋值,其中 `i` 的类型由 `auto` 关键字指定。这里用到了一种新的复合赋值运算符(参见 1.4.1 节,第 10 页)。如我们所知, `+=` 把左侧运算对象和右侧运算对象相加,结果存入左侧运算对象;类似的, `*=` 把左侧运算对象和右侧运算对象相乘,结果存入左侧运算对象。最后,第二个循环输出所有元素。

`vector` 的 `empty` 和 `size` 两个成员与 `string` 的同名成员(参见 3.2.2 节,第 78 页)功能完全一致: `empty` 检查 `vector` 对象是否包含元素然后返回一个布尔值; `size` 则返回 `vector` 对象中元素的个数,返回值的类型是由 `vector` 定义的 `size_type` 类型。



要使用 `size_type`, 需首先指定它是由哪种类型定义的。`vector` 对象的类型总是包含着元素的类型(参见 3.3 节,第 87 页):

```
vector<int>::size_type    // 正确
vector::size_type        // 错误
```

各个相等性运算符和关系运算符也与 `string` 的相应运算符(参见 3.2.2 节,第 79 页)功能一致。两个 `vector` 对象相等当且仅当它们所含的元素个数相同,而且对应位置的元素值也相同。关系运算符依照字典顺序进行比较:如果两个 `vector` 对象的容量不同,但是在相同位置上的元素值都一样,则元素较少的 `vector` 对象小于元素较多的 `vector` 对象;若元素的值有区别,则 `vector` 对象的大小关系由第一对相异的元素值的大小关系决定。

只有当元素的值可比较时, `vector` 对象才能被比较。一些类,如 `string` 等,确实定义了自己的相等性运算符和关系运算符;另外一些,如 `Sales_item` 类支持的运算已经全都罗列在 1.5.1 节(第 17 页)中了,显然并不支持相等性判断和关系运算等操作。因此,不能比较两个 `vector<Sales_item>` 对象。

计算 `vector` 内对象的索引



使用下标运算符(参见 3.2.3 节,第 84 页)能获取到指定的元素。和 `string` 一样, `vector` 对象的下标也是从 0 开始计起,下标的类型是相应的 `size_type` 类型。只要 `vector` 对象不是一个常量,就能向下标运算符返回的元素赋值。此外,如 3.2.3 节(第 85 页)所述的那样,也能通过计算得到 `vector` 内对象的索引,然后直接获取索引位置上的元素。

举个例子,假设有一组成绩的集合,其中成绩的取值是从 0 到 100。以 10 分为一个分数段,要求统计各个分数段各有多少个成绩。显然,从 0 到 100 总共有 101 种可能的成绩取值,这些成绩分布在 11 个分数段上:每 10 个分数构成一个分数段,这样的分数段有 10 个,额外还有一个分数段表示满分 100 分。这样第一个分数段将统计成绩在 0 到 9 之间的数量;第二个分数段将统计成绩在 10 到 19 之间的数量,以此类推。最后一个分数段统计满分 100 分的数量。

按照上面的描述，如果输入的成绩如下：

42 65 95 100 39 67 95 76 88 76 83 92 76 93

则输出的结果应该是：

0 0 0 1 1 0 2 3 2 4 1

结果显示：成绩在 30 分以下的没有、30 分至 39 分有 1 个、40 分至 49 分有 1 个、50 分至 59 分没有、60 分至 69 分有 2 个、70 分至 79 分有 3 个、80 分至 89 分有 2 个、90 分至 99 分有 4 个，还有 1 个是满分。

在具体实现时使用一个含有 11 个元素的 vector 对象，每个元素分别用于统计各个分数段上出现的成绩个数。对于某个成绩来说，将其除以 10 就能得到对应的分数段索引。注意：两个整数相除，结果还是整数，余数部分被自动忽略掉了。例如， $42/10=4$ 、 $65/10=6$ 、 $100/10=10$ 等。一旦计算得到了分数段索引，就能用它作为 vector 对象的下标，进而获取该分数段的计数值并加 1：

```
// 以 10 分为一个分数段统计成绩的数量：0~9, 10~19, ..., 90~99, 100
vector<unsigned> scores(11, 0); // 11 个分数段，全都初始化为 0
unsigned grade;
while (cin >> grade) {           // 读取成绩
    if (grade <= 100)             // 只处理有效的成绩
        ++scores[grade/10];      // 将对应分数段的计数值加 1
}
```

在上面的程序中，首先定义了一个 vector 对象存放各个分数段上成绩的数量。此例中，由于初始状态下每个元素的值都相同，所以我们为 vector 对象申请了 11 个元素，并把所有元素的初始值都设为 0。while 语句的条件部分负责读入成绩，在循环体内部首先检查读入的成绩是否合法（即是否小于等于 100 分），如果合法，将成绩对应的分数段的计数值加 1。

执行计数值累加的那条语句很好地体现了 C++ 程序代码的简洁性。表达式

```
++scores[grade/10]; // 将当前分数段的计数值加 1
```

等价于

```
auto ind = grade/10;           // 得到分数段索引
scores[ind] = scores[ind] + 1; // 将计数值加 1
```

上述语句的含义是：用 grade 除以 10 来计算成绩所在的分数段，然后将所得的结果作为变量 scores 的下标。通过运行下标运算获取该分数段对应的计数值，因为新出现了一个属于该分数段的成绩，所以将计数值加 1。

如前所述，使用下标的时候必须清楚地知道它是否在合理范围之内（参见 3.2.3 节，第 85 页）。在这个程序里，我们事先确认了输入的成绩确实在 0 到 100 之间，这样计算所得的下标就一定在 0 到 10 之间，属于 0 到 scores.size()-1 规定的有效范围，一定是合法的。

不能用下标形式添加元素

刚接触 C++ 语言的程序员也许会认为可以通过 vector 对象的下标形式来添加元素，事实并非如此。下面的代码试图为 vector 对象 ivec 添加 10 个元素：

```
vector<int> ivec; // 空 vector 对象
```



```
for (decltype(ivec.size()) ix = 0; ix != 10; ++ix)
    ivec[ix] = ix; // 严重错误: ivec 不包含任何元素
```

然而，这段代码是错误的：ivec 是一个空 vector，根本不包含任何元素，当然也就不能通过下标去访问任何元素！如前所述，正确的方法是使用 push_back：

105

```
for (decltype(ivec.size()) ix = 0; ix != 10; ++ix)
    ivec.push_back(ix); // 正确: 添加一个新元素，该元素的值是 ix
```



vector 对象（以及 string 对象）的下标运算符可用于访问已存在的元素，而不能用于添加元素。

提示：只能对确知已存在的元素执行下标操作！

关于下标必须明确的一点是：只能对确知已存在的元素执行下标操作。例如，

```
vector<int> ivec;           // 空 vector 对象
cout << ivec[0];           // 错误: ivec 不包含任何元素

vector<int> ivec2(10);     // 含有 10 个元素的 vector 对象
cout << ivec2[10];         // 错误: ivec2 元素的合法索引是从 0 到 9
```

试图用下标的形式去访问一个不存在的元素将引发错误，不过这种错误不会被编译器发现，而是在运行时产生一个不可预知的值。

不幸的是，这种通过下标访问不存在的元素的行为非常常见，而且会产生很严重的后果。所谓的缓冲区溢出（buffer overflow）指的就是这类错误，这也是导致 PC 及其他设备上应用程序出现安全问题的一个重要原因。



确保下标合法的一种有效手段就是尽可能使用范围 for 语句。

3.3.3 节练习

练习 3.16：编写一段程序，把练习 3.13 中 vector 对象的容量和具体内容输出出来。检验你之前的回答是否正确，如果不对，回过头重新学习 3.3.1 节（第 87 页）直到弄明白错在何处为止。

练习 3.17：从 cin 读入一组词并把它们存入一个 vector 对象，然后设法把所有词都改写为大写形式。输出改变后的结果，每个词占一行。

练习 3.18：下面的程序合法吗？如果不合法，你准备如何修改？

```
vector<int> ivec;
ivec[0] = 42;
```

练习 3.19：如果想定义一个含有 10 个元素的 vector 对象，所有元素的值都是 42，请列举出三种不同的实现方法。哪种方法更好呢？为什么？

练习 3.20：读入一组整数并把它们存入一个 vector 对象，将每对相邻整数的和输出出来。改写你的程序，这次要求先输出第 1 个和最后 1 个元素的和，接着输出第 2 个和倒数第 2 个元素的和，以此类推。

3.4 迭代器介绍



106

我们已经知道可以使用下标运算符来访问 `string` 对象的字符或 `vector` 对象的元素，还有另外一种更通用的机制也可以实现同样的目的，这就是**迭代器**（`iterator`）。在第 II 部分中将要介绍，除了 `vector` 之外，标准库还定义了其他几种容器。所有标准库容器都可以使用迭代器，但是其中只有少数几种才同时支持下标运算符。严格来说，`string` 对象不属于容器类型，但是 `string` 支持很多与容器类型类似的操作。`vector` 支持下标运算符，这点和 `string` 一样；`string` 支持迭代器，这也和 `vector` 是一样的。

类似于指针类型（参见 2.3.2 节，第 47 页），迭代器也提供了对对象的间接访问。就迭代器而言，其对象是容器中的元素或者 `string` 对象中的字符。使用迭代器可以访问某个元素，迭代器也能从一个元素移动到另外一个元素。迭代器有有效和无效之分，这一点和指针差不多。有效的迭代器或者指向某个元素，或者指向容器中尾元素的下一位置；其他所有情况都属于无效。

3.4.1 使用迭代器



和指针不一样的是，获取迭代器不是使用取地址符，有迭代器的类型同时拥有返回迭代器的成员。比如，这些类型都拥有名为 `begin` 和 `end` 的成员，其中 `begin` 成员负责返回指向第一个元素（或第一个字符）的迭代器。如有下述语句：

```
// 由编译器决定 b 和 e 的类型；参见 2.5.2 节（第 61 页）
// b 表示 v 的第一个元素，e 表示 v 尾元素的下一位置
auto b = v.begin(), e = v.end(); // b 和 e 的类型相同
```

`end` 成员则负责返回指向容器（或 `string` 对象）“尾元素的下一位置（one past the end）”的迭代器，也就是说，该迭代器指示的是容器的一个本不存在的“**尾后**（off the end）”元素。这样的迭代器没什么实际含义，仅是个标记而已，表示我们已经处理完了容器中的所有元素。`end` 成员返回的迭代器常被称作**尾后迭代器**（off-the-end iterator）或者简称为尾迭代器（end iterator）。特殊情况下如果容器为空，则 `begin` 和 `end` 返回的是同一个迭代器。



如果容器为空，则 `begin` 和 `end` 返回的是同一个迭代器，都是尾后迭代器。

一般来说，我们不清楚（不在意）迭代器准确的类型到底是什么。在上面的例子中，使用 `auto` 关键字定义变量 `b` 和 `e`（参见 2.5.2 节，第 61 页），这两个变量的类型也就是 `begin` 和 `end` 的返回值类型，第 97 页将对相关内容做更详细的介绍。

迭代器运算符

表 3.6 列举了迭代器支持的一些运算。使用 `==` 和 `!=` 来比较两个合法的迭代器是否相等，如果两个迭代器指向的元素相同或者都是同一个容器的尾后迭代器，则它们相等；否则就说这两个迭代器不相等。